



使用数据输入与输出流

北京理工大学计算机学院
金旭亮

概述



诸如`FileInputStream`、`FileOutputStream`之类的流组件，都是以单个字节为数据存取单位的，而在实际开发中，经常需要以多个字节为单位，对数据进行读取和写入。

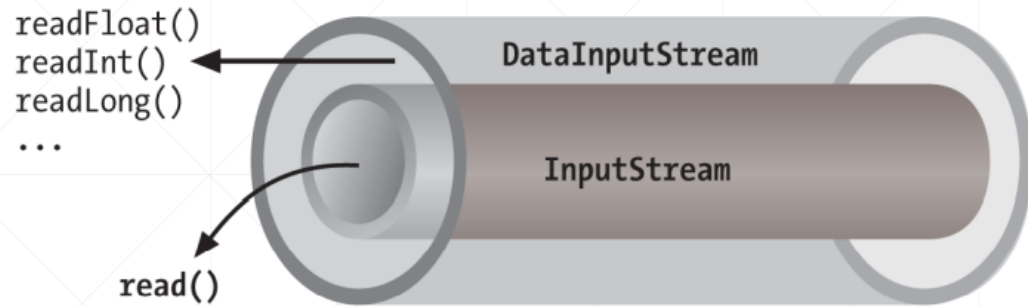


比如，一个`int`类型的数值包容4个字节。如果你需要将一个整数写入到文件中，使用`FileOutputStream`，就得多次调用`write()`方法写入4个字节到文件中，要读取也是类似的，这就给开发带来了不便。

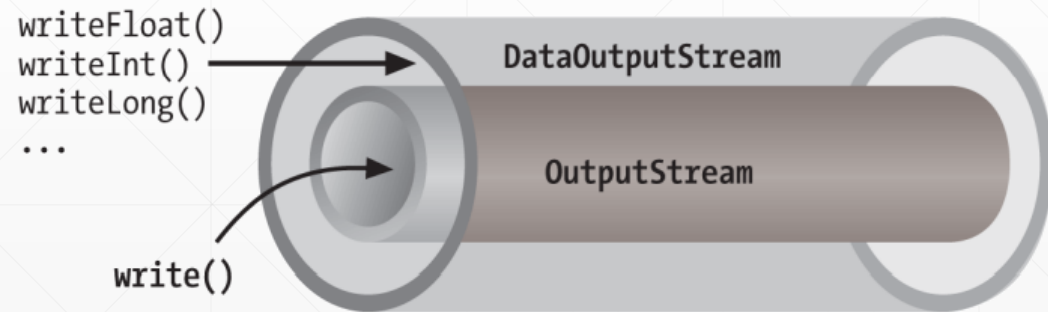


为了解决这个问题，JDK提供了`DataInputStream`和`DataOutputStream`，以便解决向流中读取和写入包容多个字节的常用数据类型值的问题。

流的嵌套

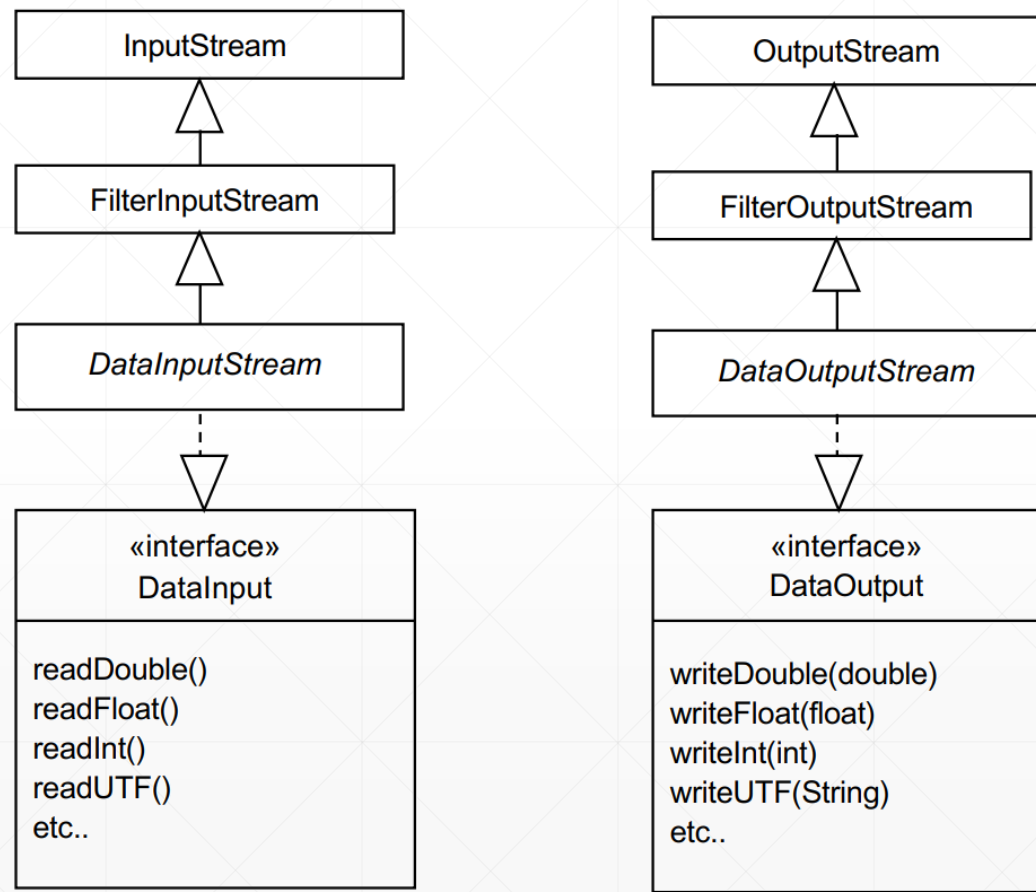


`DataInputStream`和
`DataOutputStream`自己并不负责读写数据，而是依托另外的“底层”流组件（比如`FileInputStream`）来完成读写操作。



`DataInputStream`和
`DataOutputStream`只是在其他底层流的基础上多封装了一层，让其具有一次读取多个字节并按照特定规则解析这些字节的功能（比如，将连续8个字节解析为长整数）。

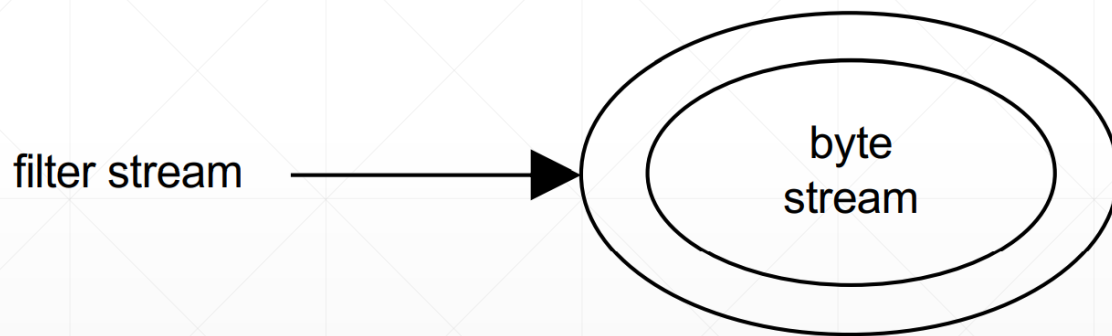
JDK中相关类型的继承与实现关系



`DataInputStream`和
`DataOutputStream`的功能由
`DataInput`和`DataOutput`接口
所封装，它本身派生自过滤器
流（`FilterInputStream`和
`FilterOutputStream`）。

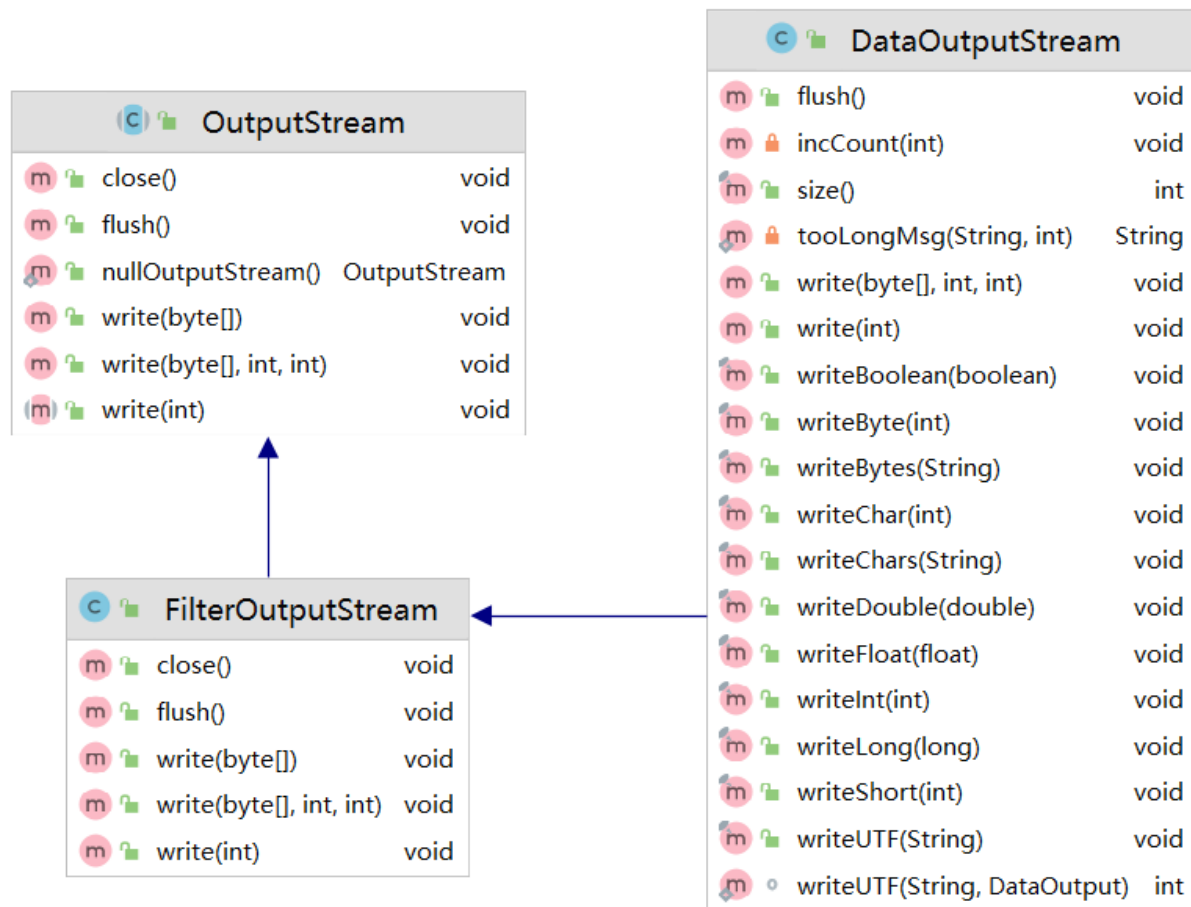
理解FilterStream

DataOutputStream派生自FilterOutputStream，利用到了它的“过滤”特性以增加向流中写入多字节数值的功能。
DataInputStream也是类似的。



FilterStream在其它流的外面包了一层，外部传入的数据先经过它检查（或加工或筛选或转换）之后，再传给内部流。因此，它多用于增强内部流的功能。

理解JDK中的设计决策



`FilterOutputStream`继承自 `OutputStream`，主要目的是提供一些可供子类重写的方法，以便在写入数据到 `OutputStream` 之前（或之后），可以进行一些额外的处理工作。

`DataOutputStream`继承自 `FilterOutputStream`，重写了基类的相应方法，同时，又扩充了写入 `int`，`char` 和 `double` 等类型数值的方法。

DataOutputStream的设计方式

FilterOutputStream的write方法，被直接委托给它所包装的OutputStream对象：

```
public class FilterOutputStream extends OutputStream {  
    /**  
     * The underlying output stream to be filtered.  
     */  
    protected OutputStream out;  
  
    @Override  
    public void write(int b) throws IOException {  
        out.write(b);  
    }  
    .....  
}
```

```
public class DataOutputStream extends FilterOutputStream  
    implements DataOutput {  
  
    public synchronized void write(int b) throws IOException {  
        out.write(b); //out在基类中定义  
        incCount(1);  
    }  
    .....  
}
```

DataOutputStream，重写了基类的write()方法，提供了“计数”功能.....

构建DataInputStream实例

代码实例

```
FileInputStream fin = new FileInputStream(文件名);  
DataInputStream din = new DataInputStream(fin);  
Double s = din.readDouble();
```

1. 上述这种代码创建的流对象“一个套一个”，最外层的流对象是程序中真正用到的流对象，最内层的流对象负责完成真正的数据存取工作。
2. 这种编程方式，在Java IO代码中非常常见。

使用示例

```
private static void writeToFile() throws IOException {  
    int i = 100;  
    double d = Math.PI;  
    try (var os = new DataOutputStream(  
        new FileOutputStream(fileName))) {  
        os.writeInt(i);  
        os.writeDouble(d);  
    }  
    System.out.println("数据写入结束");  
}
```

读写顺序要保持一致，否则，会导致数据错误。

```
private static void readFromFile() throws IOException {  
    try (var os = new DataInputStream(  
        new FileInputStream(fileName))) {  
        int i = os.readInt();  
        double d = os.readDouble();  
        System.out.printf("从文件中读取: i=%d, d=%f", i, d);  
    };  
}
```

WriteReadBinary.java

应用实例

将一个集合写入到文件中。

```
//创建一个用于测试的集合
private static Map<Integer, String> createDataCollection() {
    Map<Integer, String> data = new HashMap<>();
    data.put(1, "一");
    data.put(2, "二");
    data.put(3, "三");
    return data;
}
```

写.....

```
private static void writeToDataFile(Map<Integer, String> data) throws IOException {  
    try (var dataFile = new DataOutputStream(  
        new BufferedOutputStream(  
            new FileOutputStream( name: "data.dat")  
        )  
    )) {  
        data.forEach((index, content) -> {  
            try {  
                dataFile.writeInt(index);  
                dataFile.writeUTF(content);  
            } catch (IOException e) {  
                e.printStackTrace();  
            }  
        });  
        System.out.println("写入结束! ");  
    }  
}
```

读.....

```
private static void readCollectionFromFile() throws IOException {
    try (var reader = new DataInputStream
        (new BufferedInputStream(
            new FileInputStream( name: "data.dat")
        ))) {
        //注意一下如何判断读入结束
        boolean eof = false;
        while (!eof) {
            try {
                int index = reader.readInt();
                String content = reader.readUTF();
                System.out.println(index + ":" + content);
            } catch (EOFException exception) {
                eof = true;
            }
        }
    }
}
```

注意“读”“写”顺序要一致，否则，数据会出错。

小结



本讲详细介绍了DataInputStream和DataOutputStream的功能与设计思路，并且通过示例展示了其基本用法。



DataInputStream和DataOutputStream非常易用，学会用它并不是本讲的重点，本讲的重点在于熟悉与理解这种结构上“多层嵌套”、功能上“逐层增强”的编程模式，它在Java IO流组件代码中非常常见。